VISIÓN DE INNOVACIÓN

Contratos como motor de innovación

Automatizar pruebas de concepto y traducir intención a código desde una fuente de verdad estricta

Specs Driven Design · Octubre 2026

AI-Design-System

PROBLEMA

El cuello de botella es la interpretación

Cuando la fuente principal es visual, cada entrega vuelve a interpretar intención, comportamiento y semántica.

El resultado depende de quién lee el diseño, qué contexto conoce y cómo traduce decisiones implícitas a prototipo o código.

01

Decisiones implícitas

La apariencia no expresa todas las reglas

02

Dependencia individual

El conocimiento se concentra en pocas personas

03

Deriva entre destinos

Figma, prototipo y código pueden contar historias distintas

APUESTA

La fuente de verdad cambia de lugar

El contrato se convierte en la interfaz operativa entre una idea y sus implementaciones.

INTENCIÓN

Propósito, reglas y límites

CONTRATO

Propiedades, tokens, estados y aceptación

DESTINOS

Prueba de concepto, Figma, código y pruebas

Los componentes de Figma siguen siendo valiosos, pero pasan a ser una salida gobernada. Ya no cargan por sí solos con toda la intención del sistema.

PROCESO

Ciclo de innovación basado en contratos

La experimentación deja evidencia reutilizable en cada vuelta del ciclo.

01

Idea

Definir la hipótesis y el alcance

02

Contrato

Expresar reglas sin ambigüedad

03

Generación

Construir una prueba de concepto

04

Validación

Comprobar criterios y comportamiento

05

Aprendizaje

Registrar diferencias y decisiones

06

Nueva versión

Reutilizar el conocimiento validado

El prototipo deja de ser un punto final. Se vuelve una ejecución temporal de una especificación versionada.

MODELO OPERATIVO

Arquitectura objetivo de automatización

Una especificación estricta puede alimentar múltiples generadores sin volver a interpretar el diseño desde cero.

FUENTE DE VERDAD

Contrato MD + JSON canónico

Reglas, datos, versiones y criterios comprobables

CAPA DE AUTOMATIZACIÓN

Adaptadores y generadores

Transforman la misma especificación para cada destino

01 **Prueba de concepto**
Exploración rápida

02 **Figma**
Componentes y bindings

03 **React + TypeScript**
API y composición

04 **Pruebas y documentación**
Criterios verificables

DESCENTRALIZACIÓN

Ejecución distribuida con gobierno común

La norma permanece centralizada en el contrato. La capacidad de construir se distribuye entre equipos, herramientas y agentes.

01

Producto

Formula hipótesis y restricciones

02

Diseño

Modela experiencia, contenido y tokens

03

Desarrollo

Implementa una API alineada al contrato

04

Agentes y herramientas

Generan, comparan y validan resultados

Descentralizar no significa perder control. Significa que más actores pueden producir resultados compatibles sin depender de una interpretación privada.

Traducción limpia hacia código

El contrato reduce las decisiones que el código tendría que inferir desde una imagen o un archivo de diseño.

Propiedades	Nombres y combinaciones permitidas	**Props y tipos**
Estados	Reglas y transiciones explícitas	**Lógica verificable**
Tokens	Roles, modos y aliases	**Variables consumibles**
Aceptación	Criterios observables	**Pruebas automatizables**

Figma deja de ser el traductor obligatorio. El contrato permite reconstruir la intención en cada tecnología.

IMPACTO

Capacidad de innovación esperada

01

Más pruebas de concepto

El costo de comenzar disminuye porque la estructura básica ya está especificada.

02

Experimentos en paralelo

Distintos equipos pueden explorar destinos sin romper la fuente compartida.

03

Aprendizaje reutilizable

Las decisiones sobreviven al prototipo y alimentan la siguiente versión.

04

Herramientas reemplazables

La intención permanece estable aunque cambien Figma, el framework o el generador.

La ventaja acumulativa no proviene de producir más pantallas. Proviene de convertir cada decisión validada en infraestructura reutilizable.

RUTA PROPUESTA

Siguiente etapa de innovación

El paso siguiente consiste en demostrar que un mismo contrato puede producir resultados coherentes en más de un destino.

1. **Elegir una familia piloto** — Seleccionar un componente con variantes, tokens, contenido variable y criterios de aceptación maduros.
2. **Automatizar la prueba de concepto** — Generar una experiencia funcional desde el contrato y conservar el vínculo con sus requisitos.
3. **Traducir a código** — Producir React + TypeScript con propiedades y pruebas derivadas de la misma especificación.
4. **Medir y escalar** — Comparar tiempo, retrabajo y divergencias antes de extender el modelo a más familias.

Objetivo del piloto: validar el contrato como activo generativo, no solo como documentación.